**Name :** Shivraj Vaibhav Jagtap
**Roll No :** EN24107060
**Class :** SY - AI\&DS
**Subject :** Python for Visual AI

---

# *Assignment* 3:

---

Implement a DCT-based method to embed a robust, yet imperceptible, digital watermark onto an image to protect proprietary data. Perform the analysis of the method used.

---

## Step 1: Import Libraries

- `cv2` → Image processing
- `numpy` → Numerical operations
- `matplotlib.pyplot` → Plotting images

In [12]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

---

## Step 2: Read Image

Grayscale image:

$$I(x, y) = \text{cv2.imread}(\text{path}, 0)$$

Convert to float:

$$I_f(x, y) = \text{float32}(I(x, y))$$

In [13]:
```
img = cv2.imread('Downloads/lena.jpeg', 0)
img = np.float32(img)
```

---

## Step 3: Watermark Bit

$$W = \begin{cases} 1, & \text{watermark present} \\ 0, & \text{otherwise} \end{cases}$$

In [14]:
```
watermark = 1
```

---

## Step 4: Block Division

Divide image into $8 \times 8$ blocks:

$$B_{i,j} \in \mathbb{R}^{8\times 8}$$

In [15]:
```
block_size = 8
h, w = img.shape
```

---

## Step 5: Apply DCT

$$D_{i,j} = \mathrm{DCT}(B_{i,j})$$

In [16]:
```
watermarked_img = img.copy()
```

---

## Step 6: Embed Watermark

Modify mid-frequency coefficient:

$$D_{i,j}(4,3) = \begin{cases} D_{i,j}(4,3) + 10, & W = 1 \\ D_{i,j}(4,3) - 10, & W = 0 \end{cases}$$

### Why mid-frequency?

- Low frequency → visible distortion
- High frequency → easily removed
- Mid frequency → best balance

---

## Step 7: Inverse DCT

$$B'_{i,j} = \mathrm{IDCT}(D_{i,j})$$

---

## Step 8: Reconstruct Image

$$I'(x,y) = \bigcup B'_{i,j}$$

In [17]:
```
for i in range(0, h, block_size):
    for j in range(0, w, block_size):
        block = img[i:i+8, j:j+8]
        if block.shape == (8, 8):
            # Apply DCT
            dct_block = cv2.dct(block)
            # Embed watermark in mid-frequency coefficient
            if watermark == 1:
                dct_block[4][3] += 10
            else:
                dct_block[4][3] -= 10
# Apply IDCT
            idct_block = cv2.idct(dct_block)
# Place back
            watermarked_img[i:i+8, j:j+8] = idct_block
```

---

## Step 9: Pixel Clipping

$$I''(x,y) = \min(255, \max(0, I'(x,y)))$$

In [18]:
```
watermarked_img = np.uint8(np.clip(watermarked_img, 0, 255))
```

---

## Step 10: Output

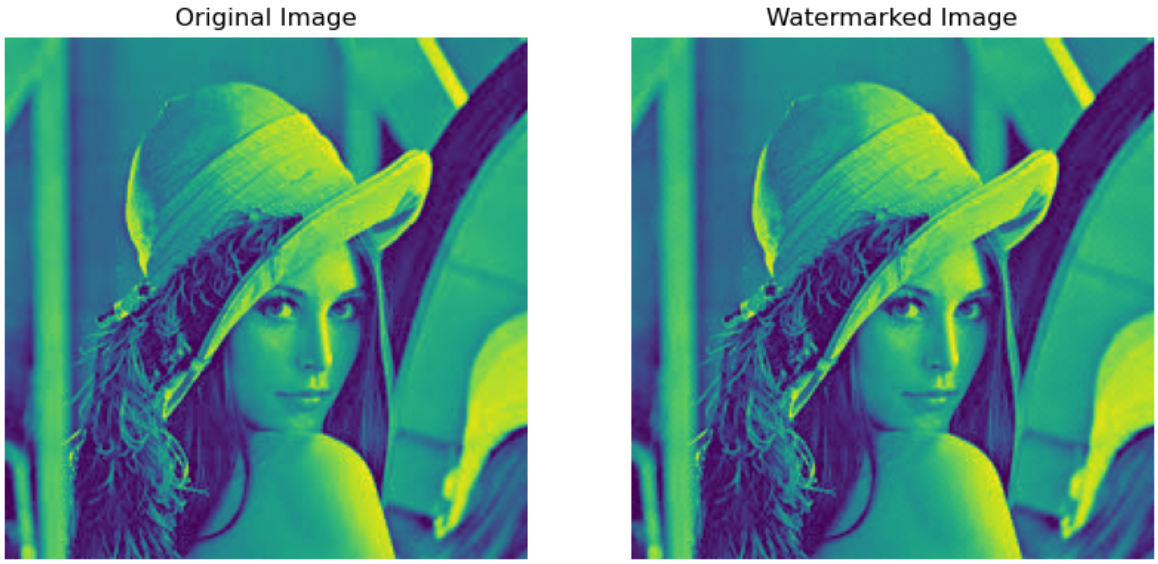
- Original Image
- Watermarked Image

In [20]:
```
cv2.imshow('Original Image', np.uint8(img))
cv2.imshow('Watermarked Image', watermarked_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
plt.figure(figsize=(10, 5)) # Optional: Adjust figure size

# Display original image
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st subplot
plt.imshow( np.uint8(img))
plt.title('Original Image')
plt.axis('off') # Optional: Hide axis ticks

# Display watermarked image
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd subplot
plt.imshow(watermarked_img)
plt.title('Watermarked Image')
plt.axis('off') # Optional: Hide axis ticks

plt.show()
```




---

## Conclusion

DCT-based watermarking embeds hidden data in frequency domain, ensuring robustness and imperceptibility. It is widely used for copyright protection and secure image transmission.